

ECE 649 — Final Project Description

Generative AI • Fall 2026

Build, evaluate, and communicate a complete generative AI system.

COURSE WEIGHT
40% of final grade

TEAM SIZE
1–4 students

PROPOSAL DUE
Oct. 7, 11:59 PM

PRESENTATIONS
Nov. 18

1. Project Purpose

The final project is the main build-and-show component of ECE 649. Your team will identify a meaningful problem, design a generative AI solution, implement a working system, evaluate it with evidence, and present what you learned to the class. The project should connect course ideas to a concrete application rather than stop at a notebook or model demo.

A successful project should demonstrate: a clear problem statement, a defensible architecture, a data or knowledge source, an evaluation method, and a discussion of limitations.

- Use at least one substantive generative AI component: e.g., an LLM/transformer, VAE, diffusion or flow model, multimodal model, agent, RAG system, generative policy, or another instructor-approved generative method.
- Go beyond “it runs.” Show what you built, what choices you made, what changed because of those choices, and how well the system works.
- Existing models, APIs, repositories, and open-source components are allowed and encouraged when properly cited. Your grade is based on your team’s contribution, integration, experimentation, evaluation, and understanding—not on the amount of code you copied.

2. Team Size and Scope

Teams may contain 1–4 students. The expected total scope scales approximately linearly with team size. A two-person team should complete roughly twice the total project work of a one-person team; a four-person team should complete roughly twice the work of a two-person team. A three-person team should fall proportionally in between.

Team	Expected scope	What scaling can look like	What does not count as scaling
1	1× baseline	One coherent end-to-end system with a focused evaluation.	Simply using a larger pretrained model.
2	≈2×	Broader system, stronger implementation, additional experiments, or a second meaningful component.	Splitting one small task in half.
3	≈3×	Multiple integrated components plus a more substantial benchmark, dataset, deployment, or ablation study.	Three people independently reproducing the same baseline.
4	≈4×	Ambitious integrated system with clear subsystems, robust evaluation, and meaningful engineering/research depth.	Adding cosmetic features without technical depth.

Important: scope is not measured only by lines of code. It may scale through technical difficulty, number of integrated components, quality and breadth of evaluation, deployment complexity, data collection, model comparison, ablation studies, or robustness testing.

3. Required Deliverables

1. **Project Proposal — due Wednesday, Oct. 7 at 11:59 PM.** Submit a concise proposal that defines what you intend to build and how you will know whether it works. One to two pages is usually sufficient.
 - Project title, team members, and team size.
 - Problem statement and motivation: who or what is the system for?
 - Generative AI component(s) and planned architecture.
 - Data, models, APIs, simulators, documents, or other knowledge sources.
 - Evaluation plan with concrete metrics, baselines, tests, or user studies.
 - Scope appropriate to team size, milestones, and division of responsibilities.
 - Expected limitations, risks, dependencies, or open questions.
2. **GitHub Repository — final version submitted with the project.** The repository is the technical artifact. It must contain working code and enough documentation for the instructor to reproduce the main result.
 - A clear README that explains the project, architecture, setup, and exact commands required to run the main demo.
 - Dependency/environment instructions and any required model or data download steps.
 - Organized, readable code with configuration separated from source when practical.
 - A reproducible evaluation script/notebook and a summary of results.
 - Citations and attribution for external models, repositories, datasets, APIs, and code.
 - No private credentials or API keys committed to the repository.
3. **In-Class Presentation — Wednesday, Nov. 18.** This is the primary graded artifact. Your team should explain the problem and technical approach, demonstrate the system, present evidence, discuss limitations, and answer questions. Presentation length will be announced once the number of teams is finalized.
 - Every team member should have a visible role in the presentation or Q&A.
 - A live demo is preferred when reliable; a short recorded fallback is strongly recommended.
 - Slides should prioritize architecture, experiments, results, failure cases, and lessons learned over background material.
4. **Peer & Self Review — after presentations.** Each student will confidentially evaluate their own contribution and the contributions of teammates. This is used to assign the individual-contribution portion of the grade and to identify major contribution imbalances.

4. Grading

The final project is worth 40% of the course grade. Within the project grade, the presentation carries the majority of the weight because the goal is not only to build a system, but to demonstrate technical understanding, evidence, and the ability to communicate your work.

Component	Weight	What is being graded
Project proposal	10%	Clarity of problem and goals; sensible architecture; evaluation plan; team-size-appropriate scope and work plan.
GitHub & reproducibility	20%	Working implementation; runnability; README/setup quality; organized code; reproducible evaluation; attribution.
Final presentation & demo	60%	Technical understanding; system/architecture; quality of demo and results; evaluation and failure analysis; communication and Q&A.
Peer & self review	10%	Individual contribution, reliability, collaboration, and ownership.

Presentation Rubric (60 points)

Criterion	Points	Strong work demonstrates...
Problem & motivation	8	A specific, important problem and a clear definition of what success means.
Technical approach & course connection	15	Architecture choices are explained, justified, and connected to generative AI concepts from the course.
Working system, demo & results	20	The system visibly works; the demo is convincing; results are substantive rather than anecdotal.
Evaluation, failure cases & limitations	10	Metrics/baselines are appropriate; failures are analyzed; limitations and tradeoffs are discussed thoughtfully.
Communication & Q&A	7	Slides are clear and efficient; all members show ownership; questions are answered with technical understanding.

Repository Rubric (20 points)

Criterion	Points	Expectation
Working & reproducible	10	The instructor can follow the documented steps and run the central system or evaluation.
Documentation	6	README explains architecture, setup, usage, data/model requirements, and results.
Code quality & attribution	4	Repository is reasonably organized, readable, and properly credits external work.

5. Expectations and Policies

- **Original contribution:** A project may begin from an existing repository, paper, model, or API, but you must make a meaningful contribution beyond reproducing the baseline. Clearly state what was inherited and what your team added.
- **Evaluation matters:** “It looks good” is not enough. Choose evidence appropriate to the system—quantitative metrics, human evaluation, latency/cost measurements, ablations, baselines, robustness tests, task success, or a combination.
- **Ambition should be controlled:** A smaller system that works end-to-end and is carefully evaluated will generally outperform an over-scoped project that is unfinished.
- **Reproducibility is part of the engineering:** If specialized hardware, paid APIs, private data, or very large downloads are required, document this clearly and provide a practical fallback demonstration when possible.
- **Individual accountability:** Most deliverables are graded as a team, but the peer/self review is individual. In cases of major, well-documented contribution imbalance, the instructor may further differentiate individual project grades.
- **Proposal approval is a scope check, not a contract:** The project may evolve. Material changes to the problem, model family, or scope should be discussed with the instructor.

6. What a Strong Project Usually Looks Like

Strong project	Weak project pattern
Builds a complete pipeline and demonstrates it on a real task.	Runs a pretrained model once and reports a few outputs.
Explains architecture choices and compares at least one meaningful alternative, baseline, or ablation.	Treats the model/API as a black box with no technical analysis.
Uses a defined evaluation set and reports successes and failures.	Selects only a few favorable examples.
Documents setup so another person can reproduce the result.	Requires undocumented environment fixes or private files.
Has scope clearly appropriate to the number of team members.	Adds people without adding meaningful scope or responsibility.

Example Proposal — “DIY Gemma Translator”

Illustrative example only. Students may choose any approved generative AI project. This example shows the expected level of specificity in a proposal and how an open-source baseline can become a course project.

Project Title	DIY Gemma Translator: An Offline Handheld Voice-to-Voice Translator
Team Size	2 students
Core GenAI Component	Gemma-based on-device translation with speech input/output
Primary Deliverable	A working Raspberry Pi handheld translator plus a reproducible evaluation

Problem Statement and Motivation

Travelers often rely on cloud-based translation apps that require connectivity and send audio or text to remote services. We propose to build a small, fully offline voice-to-voice translator that runs on consumer hardware. The goal is to explore whether an on-device generative model can provide useful multilingual translation with acceptable latency and quality on a Raspberry Pi-class device.

Baseline and Starting Point

We will begin from Google Creative Lab’s open-source [Gemma Translator repository](#), which provides an offline translator architecture for a Raspberry Pi 5. The repository uses an on-device Gemma model through LiteRT-LM, a Python backend, a React/Vite frontend optimized for a small display, microphone input, speech transcription, speech output, deployment scripts, and STL files for a physical enclosure.

Our Contribution

- Reproduce the baseline on our own Raspberry Pi 5 hardware and document a clean, repeatable setup process.
- Build the physical handheld device using a small touchscreen, microphone, speaker/headphones, and a printed or modified enclosure.
- Improve the interaction for two-person conversation, including touch-first controls and clearer turn-taking feedback.
- Create a benchmark covering at least three language pairs and multiple acoustic conditions (quiet room vs. moderate background noise).
- Measure end-to-end latency, translation quality, transcription failures, and hardware resource usage; identify where the pipeline fails and which component dominates latency/error.

- Implement at least one meaningful improvement based on the initial benchmark—for example, prompt/decoding changes, a revised language-selection interaction, audio preprocessing, or pipeline caching—and evaluate whether it improves the measured result.

System Architecture

Stage	Planned component	What we will evaluate
Speech input	Microphone + Moonshine speech recognition	Transcription errors and noise sensitivity
Translation	Gemma model running locally through LiteRT-LM	Semantic adequacy, language consistency, latency
Speech output	Local text-to-speech	Intelligibility and added latency
Interface	Small touchscreen web UI	Turn-taking, usability, recovery from errors
Hardware	Raspberry Pi 5, display, audio, enclosure	Memory use, thermal behavior, practical portability

Evaluation Plan

- Test set: 30–50 short conversational utterances per language pair, including greetings, directions, food/ordering, transportation, and common travel questions.
- Translation quality: compare generated translations against reference translations using a text metric such as chrF/BLEU plus a small human adequacy rating on a fixed sample.
- Speech robustness: report transcription success/error patterns in quiet and noisy conditions.
- Latency: measure microphone-release-to-transcript, transcript-to-translation, translation-to-speech, and total round-trip time.
- System reliability: report failure rate across repeated trials and document at least five representative failure cases.
- Ablation/improvement: compare the baseline configuration against the team's proposed improvement using the same evaluation set.

Division of Work and Milestones

Owner	Primary responsibility	Shared responsibility
Student A	Model/backend setup, inference pipeline, instrumentation and latency profiling	Benchmark design, integration, demo, presentation
Student B	Hardware/UI build, audio pipeline, enclosure/touch interaction, reproducible deployment	Benchmark design, integration, demo, presentation

Success Criteria

- The device can complete offline two-way spoken translation for the selected language pairs without internet connectivity after setup.
- The repository can be installed and launched from documented instructions on the target hardware.
- The team reports a reproducible benchmark with translation quality, latency, reliability, and failure analysis.
- At least one team-designed improvement is evaluated against the baseline rather than described only qualitatively.

Risks and Limitations

Likely limitations include model quality for some language pairs, speech-recognition errors in noisy environments, limited memory/compute on the Raspberry Pi, thermal throttling, and the difficulty of separating transcription errors from translation errors. If the full physical build becomes unreliable, the fallback demonstration will run the same

software stack on the Raspberry Pi with an external monitor/audio device while preserving the offline inference and evaluation goals.

How Scope Would Change with Team Size

- 1 student: focus on reproducing the offline translator, one or two language pairs, a compact benchmark, and one software improvement; use off-the-shelf hardware rather than a custom enclosure.
- 2 students: the proposal above—complete physical build, three language pairs, component-level profiling, and one evaluated improvement.
- 3 students: add a second substantive research/engineering axis, such as quantization/model comparison, stronger audio robustness, or a larger systematic benchmark.
- 4 students: add another major subsystem or study—for example multiple on-device model variants, comprehensive multilingual evaluation, custom hardware/UX, and a structured user study—while maintaining one integrated final system.

Reference implementation: [google-gemma/gemma-translator](https://github.com/google-gemma/gemma-translator) (accessed August 20, 2026).